HDOC Day-56

****

Write a C program that reads the size and elements of a variable-length integer array, arr[], and an integer k such that 0 < k <= size. Pass the array to a function named first_negative() using its address. Use pointers to pass the array between the calling function and first_negative(). The function first_negative() must:

- Consider every contiguous subarray of size k as a sliding window.
- Find the first negative integer in each window while moving from left to right.
- Display the first negative integer in the order of the corresponding windows.
- Display 0 if a window does not contain any negative integer.
- Separate the results by spaces.
- Display -1 if k is invalid.
- Use a nested-loop approach to solve the problem.

1. Problem Understanding

The program reads the size of an integer array, its elements, and a window size k. For every contiguous window of k elements, the function searches from left to right and displays the first negative value. If a window contains no negative value, it displays 0. If k is invalid, it displays -1. The array is passed to first_negative() using its address and processed using pointer notation.

2. Standard Test Cases

Test Case	Size (n)	Array Input	k	Expected Output
1	8	12 -1 -7 8 -15 30 16 28	3	-1 -1 -7 -15 -15 0
2	6	1 2 -3 4 5 -6	2	0 -3 -3 0 -6
3	4	1 2 3 4	2	0 0 0
4	3	1 2 3	4	-1
5	5	-5 2 -1 4 -3	1	-5 0 -1 0 -3

3. Algorithm

Main Algorithm

1. Start.
2. Read the size n.
3. Declare the variable-length array arr[n].
4. Read n array elements.
5. Read k.
6. Pass arr, n and k to first_negative().
7. Stop.

Function: first_negative()

1. If k <= 0 or k > n, display -1 and return.
2. For each starting index i from 0 to n-k, set found = 0.

3. Using an inner loop, check elements from i to i+k-1.

4. If a negative element is found, display it, set found = 1, and break the inner loop.

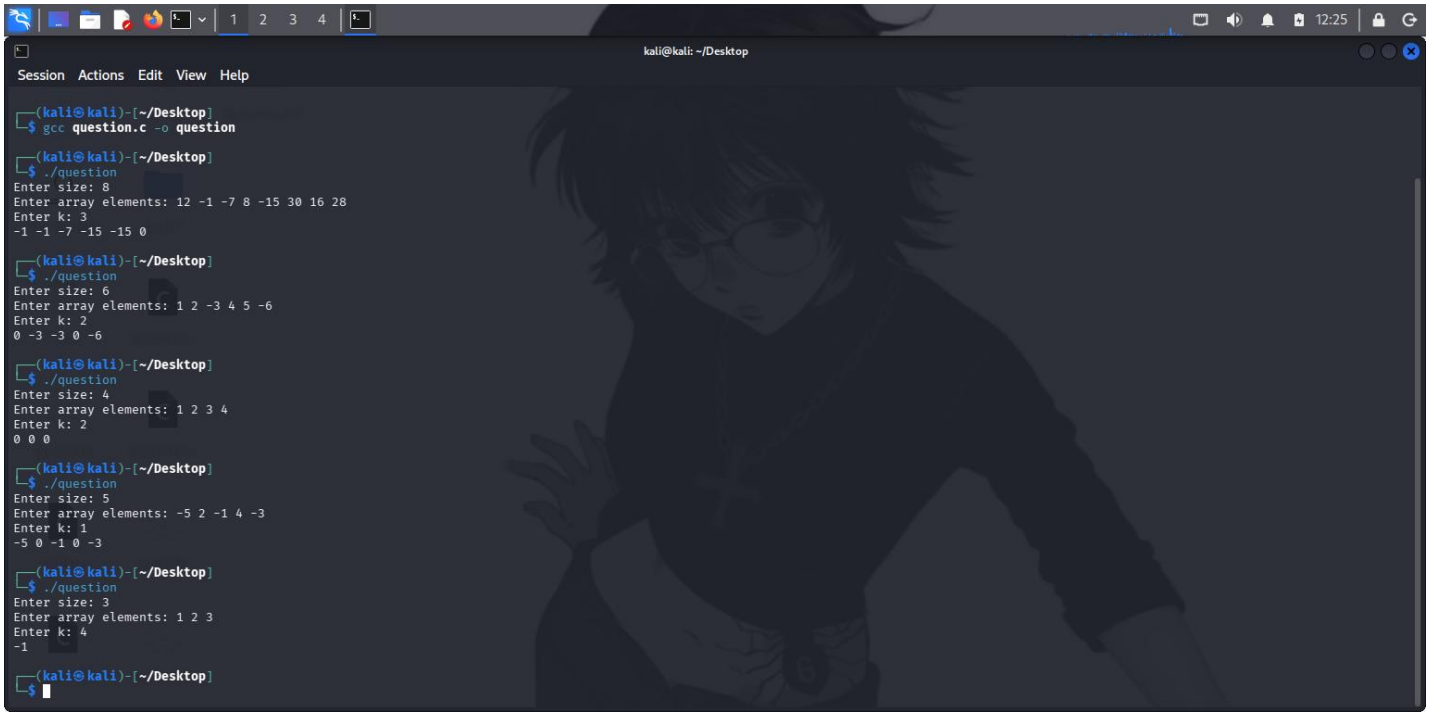
5. If no negative element is found in that window, display 0.

6. Continue until all windows are processed.

4. C Program

```
#include <stdio.h>
void first_negative(int *arr, int n, int k)
{
  int i, j, found;
  if(k <= 0 || k > n)
  {
    printf("-1\n");
    return;
  }
  for(i = 0; i <= n - k; i++)
  {
    found = 0;
    for(j = i; j < i + k; j++)
    {
      if(*(arr + j) < 0)
      {
        printf("%d", *(arr + j));
        found = 1;
        break;
      }
    }
    if(found == 0)
      printf("0");
    if(i < n - k)
      printf(" ");
  }
  printf("\n");
}
int main()
{
  int n, i, k;
  printf("Enter size: ");
  scanf("%d", &n);
  int arr[n];
  printf("Enter array elements: ");
  for(i = 0; i < n; i++)
    scanf("%d", &arr[i]);
  printf("Enter k: ");
  scanf("%d", &k);
  first_negative(arr, n, k);
  return 0;
}
```

5. Compilation and Execution Evidence



6. Verification of All Test Cases

Test Case	Size (n)	Array Input	k	Actual Result	Status
1	8	12 -1 -7 8 -15 30 16 28	3	-1 -1 -7 -15 -15 0	Pass
2	6	1 2 -3 4 5 -6	2	0 -3 -3 0 -6	Pass
3	4	1 2 3 4	2	0 0 0	Pass
4	3	1 2 3	4	-1	Pass
5	5	-5 2 -1 4 -3	1	-5 0 -1 0 -3	Pass